

Dokumentacja projektu

Topological Visual Toolkit

Julia Bugajska Filip Jastrzębski Krzysztof Korzeniewski

Repozytorium projektu:

<https://github.com/DevFifi/Topological-Visual-Toolkit/>

Działająca aplikacja: <https://topological-visual-toolkit.streamlit.app>

1 Cel projektu

Celem projektu było przygotowanie aplikacji z wygodnym interfejsem, która realizuje sześć zadań z przedmiotu *Elementy topologii stosowanej*. Program został napisany jako aplikacja Streamlit. Poszczególne moduły pozwalają liczyć odległości w przestrzeniach metrycznych, odległości Czebyszewa funkcji, wielomiany Bernsteina oraz przybliżać obrazy i przeciwobrazy zbiorów.

W projekcie przyjęto podejście praktyczne: tam, gdzie da się otrzymać wynik symboliczny lub dokładny zapis, program próbuje go pokazać. W przypadkach wymagających metod numerycznych wynik jest opisany jako przybliżony. Jest to ważne szczególnie przy supremach i rysowaniu przeciwobrazów, ponieważ dla ogólnych wzorów często trzeba korzystać z metod przybliżonych.

2 Ogólne działanie programu

Aplikacja ma sześć widocznych podprogramów, wybieranych z panelu bocznego:

1. *Skończone przestrzenie metryczne,*
2. *Odległość supremum na przedziale,*
3. *Odległość supremum na prostokącie,*
4. *Aproksymacja Bernsteina,*
5. *Przeciwobraz funkcji skalarnej,*
6. *Odwzorowania wektorowe.*

Zgodność z treścią zadań

Zadanie	Realizacja w aplikacji
22	Skończone przestrzenie metryczne: macierz odległości, średnica i odległość między zbiorami w $\mathbb{R}^n$.
19	Supremum na przedziale: $d_\infty(f, g)$ na $[a, b]$ oraz punkty osiągnięcia maksimum.
20	Supremum na prostokącie: $d_\infty(f, g)$ na prostokącie w $\mathbb{R}^2$.
20	Aproksymacja Bernsteina: wykres f , wykres $B_n(f)$, błąd i animacja.
5	Funkcja $\mathbb{R}^2 \rightarrow \mathbb{R}$: sprawdzenie punktu i rysunek $f^{-1}(A)$.
6	Odwzorowania wektorowe: rysunki $\Phi(C)$ oraz $\Phi^{-1}(B)$.

We wszystkich modułach używany jest wspólny sposób wpisywania wzorów. Użytkownik może pisać zarówno zwykłe wyrażenia, np. `sin(x)+x^2`, jak i zapis podobny do LaTeX-a, np. `\frac{1}{2}`, `\sqrt{x}`, `\pi`, `e^x`. Po rozpoznaniu wzoru program pokazuje podgląd w postaci matematycznej. W panelu bocznym znajduje się też klawiatura matematyczna, która wstawia typowe symbole i operatory do aktywnego pola tekstowego.

Program ma także lokalną pamięć przykładów. Każdy moduł korzysta ze swojej historii, dzięki czemu przykłady z jednego zadania nie mieszają się z przykładami z innego. Wyjątkiem są pola tego samego typu w ramach jednego modułu, np. zbiory E i F w module metrycznym mogą korzystać z tej samej listy zapisanych zbiorów punktów.

3 Wspólne techniki

Do obliczeń symbolicznych i numerycznych używane są biblioteki Pythona: przede wszystkim SymPy, NumPy, SciPy oraz Plotly. SymPy służy do parsowania i upraszczania wzorów, NumPy do szybkich obliczeń tablicowych, SciPy do optymalizacji numerycznej, a Plotly do interaktywnych wykresów. Funkcje wpisane przez użytkownika są zamieniane na funkcje numeryczne przez `lambdify`; wartości nieskończone i nieokreślone są odrzucane, żeby pojedynczy błędny punkt siatki nie psuł całego wyniku.

Parser wzorów obsługuje podstawowe działania, nawiasy, potęgowanie, pierwiastki, ułamki, stałe e i π , funkcje trygonometryczne, funkcje wykładnicze i logarytmiczne. Parser zbiorów rozpoznaje między innymi przedziały, zbiory skończone, prostokąty, relacje oraz operacje logiczne sumy i przecięcia.

Mapa użytych technik

Część programu	Główne techniki
Parsery	normalizacja zapisu, rekurencyjne dzielenie po operatorach najwyższego poziomu, SymPy
Metryki	rachunek symboliczny, blokowe obliczenia NumPy, wzory dla pudełek i przedziałów
Suprema	siatka punktów, lokalna optymalizacja SciPy, osobne badanie brzegów
Bernstein	stabilna ewaluacja przez logarytmy, próbkowanie $f(k/n)$, numeryczny błąd supremum
Rysunki zbiorów	maski logiczne na siatce, kontury Plotly, linie ciągłe i przerwane dla brzegu

3.1 Obsługiwane rodzaje wejść

W polach funkcji można wpisywać na przykład:

x^2+y^2 , $\sqrt{x}$, e^x , $\sin(\pi x)$, $\frac{1+x}{2}$.

W module odwzorowań wektorowych osobno wpisuje się dwie współrzędne odwzorowania: $\Phi_1(x, y)$ i $\Phi_2(x, y)$.

Punkty i zbiory skończone można podawać w postaci list:

$(0,0)$, $(1,1)$, $[(0,0), (e, \sqrt{2})]$, $\{(0,0), (\pi, 1/2)\}$.

W module metrycznym program sprawdza wymiar punktów i w razie zmiany n dopasowuje liczbę współrzędnych.

Zbiory w $\mathbb{R}$ mogą być przedziałami i zbiorami skończonymi:

$[0,1]$, $(0,1]$, $(-\infty, 0)$, $\{0,1,\pi\}$.

Można je również łączyć przez sumę i przecięcie, np.

$[0,1] \cup \{2\}$, $[0,2] \cap (1,3)$.

Zbiory w $\mathbb{R}^2$ mogą być prostokątami, relacjami albo zbiorami punktów. Przykładowe poprawne zapisy to:

$[-1,1] \times [-2,2]$, $[-1,1] \times [-2,2]$,
 $x^2+y^2 \leq 1$, $1/4 < x^2+y^2 \leq 1$,
 $\{(0,0), (e, \sqrt{2})\}$.

Warunki można łączyć operatorami $\cap$, $\cup$, $\land$, $\lor$, a także zapisami **and** i **or**. W zbiorach docelowych dla odwzorowania Φ można używać zmiennych u, v zamiast x, y .

W module metrycznym, oprócz skończonych list punktów, obsługiwane są też proste zbiory pudełkowe i ich kombinacje:

$[-1,1] \times [0,2]$, $[-1,1] \times \{0,1\} \times [0,2]$.

Można używać także przecięć i sum takich składników.

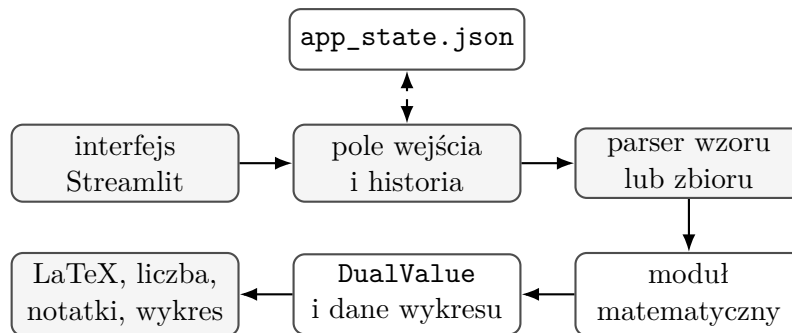
Przy rysowaniu zbiorów w $\mathbb{R}^2$ stosowana jest siatka punktów oraz dodatkowe wyznaczanie brzegu przez kontury funkcji. Dzięki temu kształty opisane nierównościami, np. krzywe algebraiczne, są wyświetlane znacznie dokładniej niż przy samym kolorowaniu pojedynczych pikseli siatki. Brzegi zbiorów otwartych są oznaczane linią przerywaną, a domkniętych linią ciągłą, o ile program jest w stanie to rozpoznać z wpisanego warunku.

3.2 Architektura rozwiązania

Kod został podzielony na trzy główne części. Warstwa **ui** odpowiada za formularze, podglądy LaTeX, historię wejść i wykresy. Warstwa **core** zawiera parsery, formatowanie wyników, bezpieczną ewaluację numeryczną i zapis stanu aplikacji. Warstwa **math_modules** zawiera właściwe algorytmy matematyczne używane przez sześć podprogramów.

Przepływ danych w typowym module

Użytkownik wpisuje wzór albo zbiór. Tekst jest normalizowany i parsowany. Jeżeli zapis jest poprawny, program pokazuje podgląd matematyczny. Po kliknięciu przycisku obliczeń funkcje są zamieniane na bezpieczne funkcje numeryczne, wykonywany jest algorytm modułu, a wynik trafia do wspólnego formatu zawierającego część dokładną, numeryczną, opis metody i ewentualne uwagi.



Rysunek 1: Uproszczony przepływ danych w aplikacji.

```

input_text
-> normalizacja zapisu LaTeX-like
-> parse_expr z dozwolonymi symbolami i funkcjami
-> podgląd LaTeX albo komunikat błędu
-> obliczenia symboliczne i/lub numeryczne
-> DualValue: exact, exact_latex, numeric, method, notes
-> wynik w interfejsie
  
```

Algorytm 1: Ogólny schemat działania pól matematycznych

3.3 Parser wyrażeń

Parser wyrażeń przyjmuje praktyczny podzbiór zwykłego zapisu matematycznego i zapisu podobnego do LaTeX-a. Przed przekazaniem tekstu do SymPy wykonywana jest normalizacja: potęgowanie $^$ jest traktowane jak potęgowanie, $\frac{a}{b}$ jest zamieniane na iloraz, $\sqrt{x}$ na pierwiastek, π na π , a zapis e na stałą Eulera. Obsługiwane są też nawiasy $\left$, $\right$, funkcje trygonometryczne, logarytmy, wartość bezwzględna zapisana jako $|x-y|$ oraz mnożenie domyślne, np. $2x$.

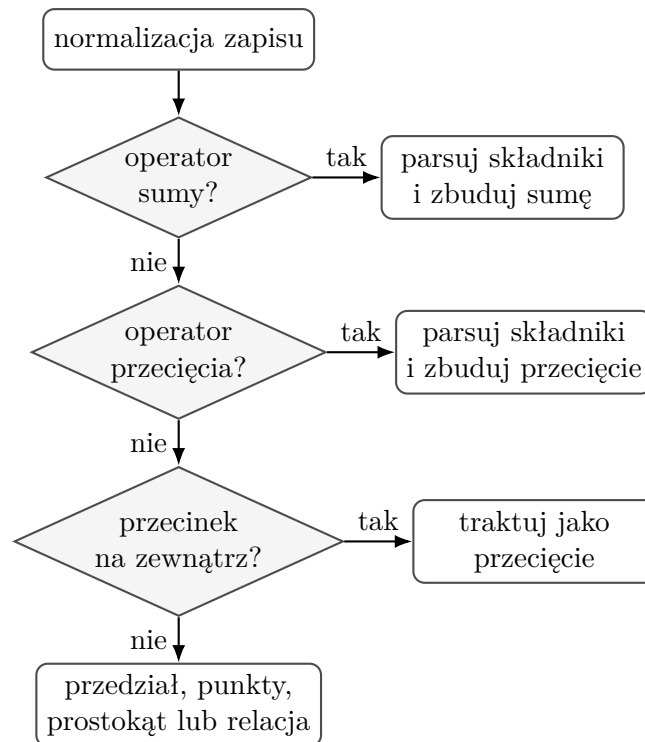
```

parse_expression(text):
    if text is empty:
        return error
    text = replace_unicode_symbols(text)
    text = replace_latex_frac(text)
    text = replace_latex_sqrt(text)
    text = replace_latex_functions(text)
    text = replace_absBars(text)
    namespace = {x,y,u,v,x1..x200,y1..y200,e,pi,sin,cos,...}
    return sympy.parse_expr(text, implicit_multiplication, convert_xor)
  
```

Algorytm 2: Normalizacja i parsowanie wyrażenia

3.4 Parser zbiorów

Parser zbiorów działa rekurencyjnie. Najpierw normalizuje operatory, np. $\cap$, $\land$, and do przecięcia oraz $\cup$, $\lor$, or do sumy. Następnie dzieli zapis tylko po operatorach znajdujących się na najwyższym poziomie nawiasów. Dzięki temu wyrażenie z przecinkiem w punkcie albo przedziale nie jest błędnie rozdzielane.



Rysunek 2: Schemat rekurencyjnego parsera zbiorów.

```

parse_set(text):
    text = normalize_set_operators(text)
    if top_level_OR_exists(text):
        return union(parse_set(left), parse_set(right))
    if top_level_AND_exists(text):
        return intersection(parse_set(left), parse_set(right))
    if top_level_commas_exist(text):
        return intersection(parse_set(part_1), ..., parse_set(part_k))
    return parse_atom(text)

parse_atom(text):
    try interval, finite set, rectangle, finite point set
    try chained relation: a < expr <= b
    try single relation: lhs <= rhs
  
```

Algorytm 3: Rekurencyjne parsowanie zbioru

Dłaczego dzielenie po najwyższym poziomie jest ważne

Zapis $\{(0,0), (1,1)\}$ zawiera przecinki, ale nie oznacza przecięcia warunków. Program śledzi nawiasy okrągłe, kwadratowe i klamrowe, więc rozdziela tylko te operatory, które naprawdę stoją między składnikami zbioru.

3.5 Dokładność, format wyników i bezpieczeństwo obliczeń

Wyniki są przechowywane w małej strukturze `DualValue`. Może ona zawierać dokładny zapis symboliczny, zapis LaTeX, wartość numeryczną, opis metody oraz notatki o dokładności. Dzięki temu każdy moduł może rozróżnić wynik dokładny od wyniku numerycznego, zamiast wyświetlać wszystkie liczby tak samo.

Dokładne wyrażenia są upraszczane kilkoma metodami SymPy: `simplify`, `radsimp`, `sqrtdenest`, rozwijanie pierwiastków i faktoryzacja. Program wybiera wariant o najczytelniejszym zapisie według prostej funkcji oceny, która karze między innymi niepotrzebne kwadraty nawiasów.

```
simplify_exact(expr):
    candidates = [
        expr,
        simplify(expr),
        radsimp(expr),
        sqrtdenest(expr),
        expand_roots(expr),
        factor(expand(simplify(expr)))
    ]
    remove failed transformations
    return candidate with shortest readable representation
```

Algorytm 4: Dobór czytelniejszej postaci dokładnej

Funkcje numeryczne są tworzone przez `lambdify`, ale są opakowane w funkcje bezpieczne. Jeżeli w jakimś punkcie pojawi się dzielenie przez zero, logarytm z liczby niedodatniej albo przepełnienie, wynik w tym punkcie jest zamieniany na `NaN`. Algorytmy szukające maksimum albo rysujące zbiory ignorują takie punkty.

3.6 Pamięć przykładów i stan aplikacji

Historia wejść jest zapisywana w pliku JSON. Każdy moduł ma osobne kategorie historii, dlatego przykłady funkcji z modułu Bernsteina nie mieszają się z przykładami zbiorów z modułu odwzorowań. Przy polach tego samego typu w jednym module historia jest wspólna, np. zbiory E i F w module metrycznym korzystają z tej samej listy zapisanych zbiorów punktów.

Program ma też plik stanu początkowego z przykładami. Przy pierwszym uruchomieniu albo przy przywracaniu stanu aplikacji dane z tego pliku są wczytywane do bieżącej pamięci. Przy aktualizacjach przykłady są scalane po identyfikatorze i po surowej wartości wpisu, żeby nie tworzyć zbędnych duplikatów.

4 Podprogram 1: skończone przestrzenie metryczne

4.1 Opis zadania

Dane są: liczba $n \in \mathbb{N} \setminus \{0\}$, metryka d w przestrzeni $\mathbb{R}^n$ oraz niepusty skończony zbiór

$$E = \{p_1, \dots, p_s\} \subseteq \mathbb{R}^n.$$

Program wypisuje macierz odległości

$$D(p_1, \dots, p_s) = \left[d(p_i, p_j) \right]_{i,j=1}^s$$

oraz średnicę zbioru

$$\text{diam}(E) = \max_{p,q \in E} d(p, q).$$

Dla dwóch niepustych zbiorów skończonych $E, F \subseteq \mathbb{R}^n$ program oblicza także

$$\text{dist}(E, F) = \inf \{ d(p, q) : p \in E, q \in F \}.$$

Ponieważ zbiory są skończone, infimum jest w tym przypadku minimum.

4.2 Metoda

Dostępne metryki

W programie dostępne są między innymi następujące metryki:

$$\delta(x, y) = \begin{cases} 0, & x = y, \\ 1, & x \neq y, \end{cases}$$

$$d_H(x, y) = \# \{ i : x_i \neq y_i \},$$

$$d_p(x, y) = \begin{cases} (\sum_{i=1}^n |x_i - y_i|^p)^{1/p}, & p \geq 1, \\ \sum_{i=1}^n |x_i - y_i|^p, & 0 < p < 1, \end{cases}$$

oraz

$$d_\infty(x, y) = \max_{1 \leq i \leq n} |x_i - y_i|.$$

Można również wpisać własny wzór metryki, np. sumę po współrzędnych. Program sprawdza poprawność wymiarów punktów, niepustość zbiorów oraz przypadki, w których wzór daje wartości nienumeryczne.

Obliczanie wyników

Dla zwykłych skończonych list punktów program tworzy symetryczną macierz odległości. Wartości na przekątnej są równe 0, a dla par $i < j$ odległość jest liczona tylko raz i przepisywana do komórki symetrycznej. Jeżeli współrzędne punktów są symboliczne, np. zawierają $\sqrt{2}$, π albo e , program najpierw próbuje zachować dokładny zapis, a dopiero potem oblicza wartość dziesiętną.

Średnica zbioru skończonego jest maksimum po wszystkich parach punktów z E , a odległość dwóch zbiorów skończonych jest minimum po parach $(p, q) \in E \times F$. Dla bardzo dużych zbiorów albo dużego wymiaru program przełącza się na szybszą ścieżkę numeryczną: punkty są zamieniane na tablice NumPy, a odległości są liczone blokami. Dzięki temu nie trzeba trzymać w pamięci całej ogromnej macierzy odległości naraz.

```

distance_matrix(E, d):
    for i = 1..s:
        D[i,i] = 0
    for i = 1..s:
        for j = i+1..s:
            value = d(p_i, p_j)
            D[i,j] = value
            D[j,i] = value
    return D

diameter(E, d):
    best = 0
    for every unordered pair (p_i, p_j):
        best = max(best, d(p_i, p_j))
    return best

```

Algorytm 5: Macierz odległości i średnica zbioru skończonego

```

large_set_distance(E, F, d):
    convert E and F to numeric arrays
    split arrays into blocks of about 160 points
    for every block pair:
        compute pairwise distances by NumPy broadcasting
        update current minimum or maximum
    return best value and realizing pair

```

Algorytm 6: Szybka ścieżka dla dużych zbiorów

Jeżeli użytkownik poda zbiór w postaci przedziałowej lub pudełkowej, program nie wypisuje macierzy punktowej. W takim przypadku średnica i odległość są liczone z końców odpowiednich przedziałów. Dla odległości między pudełkami używany jest odstęp domknięć przedziałów w każdej współrzędnej, a dla średnicy największa możliwa różnica końców. Przy otwartych brzegach wynik może być supremum albo infimum nieosiąganym przez konkretne punkty, więc program opisuje to w komunikacie.

Zbiory pudełkowe w module metrycznym

Dla produktu przedziałów program sprowadza odległość do listy różnic w kolejnych współrzędnych. Dla $\text{dist}(E, F)$ używany jest odstęp między przedziałami, a dla $\text{diam}(E)$ największa różnica między ich końcami. Następnie ta lista różnic trafia do wybranej metryki: d_1 , d_2 , d_∞ , d_p , Hamminga albo dyskretnej.

4.3 Przykład

Dla $n = 2$, metryki euklidesowej i zbioru

$$E = \{(0, 0), (1, 0), (0, 1), (1, 1)\}$$

macierz odległości ma na przekątnej zera, a największa odległość wynosi $\sqrt{2}$. Program pokazuje wartości w czytelnej postaci LaTeX, np. $\sqrt{2}$ zamiast długiego zapisu technicznego.

Macierz odległości D(e_i, e_j)

	(0, 0)	(1, 0)	(0, 1)	(1, 1)
(0, 0)	0 ~ 0.0	1 ~ 1.0	1 ~ 1.0	$\sqrt{2}$ ~ 1.4142135623730951
(1, 0)	1 ~ 1.0	0 ~ 0.0	$\sqrt{2}$ ~ 1.4142135623730951	1 ~ 1.0
(0, 1)	1 ~ 1.0	$\sqrt{2}$ ~ 1.4142135623730951	0 ~ 0.0	1 ~ 1.0
(1, 1)	$\sqrt{2}$ ~ 1.4142135623730951	1 ~ 1.0	1 ~ 1.0	0 ~ 0.0

Średnica diam(E)

Wynik

Dokładnie

$\sqrt{2}$
~ 2

~ 1.4142135623730951

Para realizująca

$e_i = (0, 0), \quad e_j = (1, 1)$

Macierz odległości E x F

	(e, π)	($\sqrt{5}, 1 + e$)
(0, 0)	$\sqrt{e^2 + \pi^2}$ ~ 4.154354402313314	$\sqrt{2e + 6 + e^2}$ ~ 4.338650049936202
(1, 0)	$\sqrt{-2e + 1 + e^2 + \pi^2}$ ~ 3.580795560081854	$\sqrt{-2\sqrt{5} + 2e + 7 + e^2}$ ~ 3.9183521792775546
(0, 1)	$\sqrt{-2\pi + 1 + e^2 + \pi^2}$ ~ 3.4605599536549603	$\sqrt{5 + e^2}$ ~ 3.519809099785193
(1, 1)	$\sqrt{-2\pi - 2e + 2 + e^2 + \pi^2}$ ~ 2.745707838777158	$\sqrt{-2\sqrt{5} + 6 + e^2}$ ~ 2.98612125405702

Odległość dist(E, F)

Wynik

Dokładnie

$\sqrt{-2\pi - 2e + 2 + e^2 + \pi^2}$
~ 2.745707838777158

~ 2.745707838777158

Para realizująca

$e = (1, 1) \in E, \quad f = (e, \pi) \in F$

Macierz odległości i średnica zbioru.

Odległość między dwoma zbiorami punktów.

Rysunek 3: Dwa przykładowe wyniki działania modułu.

5 Podprogram 2: odległość supremum na przedziale

5.1 Opis zadania

Dany jest ograniczony przedział domknięty

$$J = [a, b] \subseteq \mathbb{R}$$

oraz funkcje ciągłe $f, g : J \rightarrow \mathbb{R}$. Program oblicza odległość Czebyszewa

$$d_{\infty}(f, g) = \sup_{x \in J} |f(x) - g(x)|.$$

Dodatkowo rysowany jest wykres pokazujący punkty, w których wartość $|f(x) - g(x)|$ jest równa lub bardzo bliska otrzymanemu supremum.

5.2 Metoda

Program najpierw bada funkcję

$$h(x) = |f(x) - g(x)|.$$

Sprawdzone są końce przedziału oraz punkty znalezione metodami numerycznymi. Najpierw wartości funkcji są liczone na gęstej siatce, a następnie najlepsze kandydаты są poprawiane lokalnie metodą optymalizacji jednej zmiennej. Dla prostszych wzorów program dodatkowo rozważa pochodną funkcji $(f - g)^2$, szuka punktów krytycznych i porównuje je z końcami przedziału. Jeżeli ta część symboliczna powiedzie się i zgadza się z wynikiem numerycznym, wynik jest pokazany także dokładnie.

```
supremum_interval(f, g, [a,b]):  
    h(x) = abs(f(x) - g(x))  
    evaluate h on a dense grid  
    candidates = grid maxima + endpoints  
    for the best local candidates:  
        improve candidate by bounded scalar optimization  
    numeric_result = best value found  
  
    if expression is not too complicated:  
        solve derivative of (f-g)^2  
        check roots inside [a,b] and endpoints  
        if symbolic value agrees with numeric_result:  
            return exact and numeric result  
    return numeric result
```

Algorytm 7: Supremum na przedziale

Dlaczego używamy $(f - g)^2$?

Maksima funkcji $|f - g|$ są takie same jak maksima $(f - g)^2$, a pochodna kwadratu jest zwykle wygodniejsza symbolicznie niż pochodna wartości bezwzględnej.

Przyjęta klasa funkcji to funkcje opisane jawnym wzorem, które da się obliczać na przedziale J i które poza ewentualnymi punktami wykluczonymi przez dziedzinę nie powodują wartości nieskończonych lub NaN. Użytkownik powinien dobrać przedział tak, aby wpisane funkcje rzeczywiście były na nim ciągłe.

5.3 Przykład

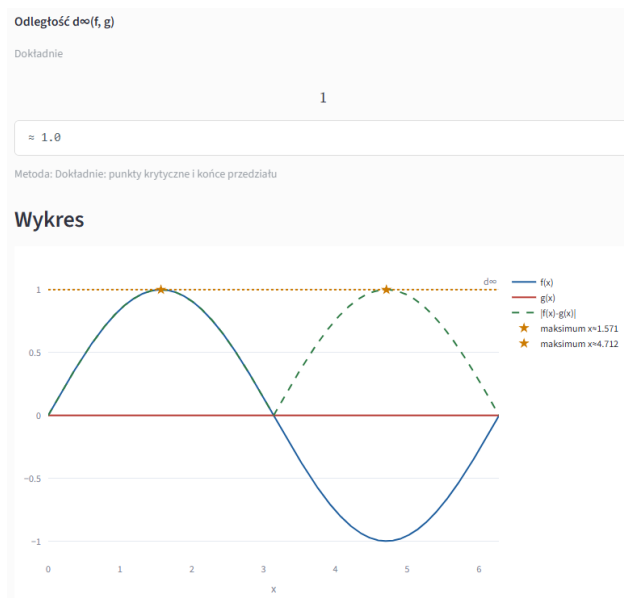
Dla

$$f(x) = \sin x, \quad g(x) = 0, \quad J = [0, 2\pi]$$

otrzymujemy

$$d_{\infty}(f, g) = 1.$$

Maksimum jest osiągnięte w punktach, w których $\sin x = \pm 1$.



Rysunek 4: Odległość Czebyszewa dwóch funkcji na przedziale.

6 Podprogram 3: odległość supremum na prostokącie

6.1 Opis zadania

Dany jest prostokąt domknięty

$$P = [a, b] \times [c, d] \subseteq \mathbb{R}^2$$

oraz funkcje ciągłe $f, g : P \rightarrow \mathbb{R}$. Program wyznacza przybliżoną odległość

$$d_\infty(f, g) = \sup_{(x,y) \in P} |f(x, y) - g(x, y)|.$$

6.2 Metoda

Podobnie jak w przypadku jednowymiarowym rozważana jest funkcja

$$h(x, y) = |f(x, y) - g(x, y)|.$$

Program sprawdza narożniki, brzegi oraz punkty wewnątrz prostokąta. Wewnątrz prostokąta najpierw liczona jest siatka wartości, potem kilka najlepszych punktów startowych jest poprawianych metodą L-BFGS-B z ograniczeniami do danego prostokąta. Brzegi są traktowane osobno jako cztery zadania jednowymiarowe. Ostateczny wynik jest największą wartością znalezioną w tych krokach.

```
supremum_rectangle(f, g, P):  
    h(x,y) = abs(f(x,y) - g(x,y))  
    evaluate h on a rectangular grid  
    starts = several best grid points + corners  
    for start in starts:  
        improve by L-BFGS-B with bounds of P  
  
    for each of four edges of P:  
        solve one-dimensional maximum on the edge  
  
    also check four corners  
    return the largest finite value found
```

Algorytm 8: Supremum na prostokącie

Rola brzegów

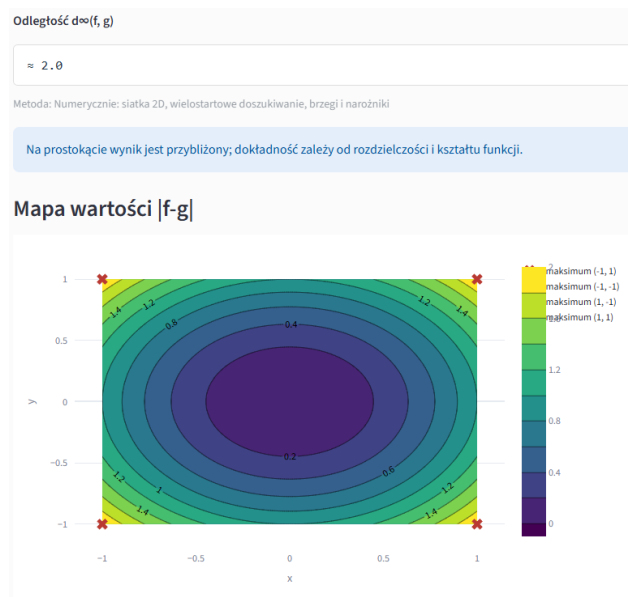
Sama optymalizacja z punktów wewnętrznych może przeoczyć maksimum leżące na brzegu. Dlatego program osobno maksymalizuje funkcję na bokach prostokąta i osobno sprawdza narożniki.

6.3 Przykład

Dla

$$f(x, y) = x^2 + y^2, \quad g(x, y) = 0, \quad P = [-1, 1] \times [-1, 1]$$

największa wartość występuje w narożnikach prostokąta i wynosi 2.



Rysunek 5: Odległość Czebyszewa funkcji na prostokacie.

7 Podprogram 4: aproksymacja Bernsteina

7.1 Opis zadania

Dla funkcji ciągłej $f : [0, 1] \rightarrow \mathbb{R}$ program pokazuje, jak wielomiany Bernsteina przybliżają funkcję f . Wielomian stopnia n ma postać

$$B_n(f)(x) = \sum_{k=0}^n f\left(\frac{k}{n}\right) \binom{n}{k} x^k (1-x)^{n-k}.$$

Aplikacja rysuje wykresy f oraz $B_n(f)$, oblicza przybliżoną odległość

$$d_\infty(f, B_n(f)) = \sup_{x \in [0,1]} |f(x) - B_n(f)(x)|$$

oraz tworzy animację pokazującą zmianę wykresu $B_n(f)$ przy wzroście n .

7.2 Metoda

Dla małych stopni program może pokazać pełną postać symboliczną wielomianu. W kodzie jest to ograniczone do niewielkich n , ponieważ pełny wzór bardzo szybko staje się nieczytelny. Dla większych stopni program wyświetla tylko kilka pierwszych składników sumy i wielokropek.

Numerycznie wielomian jest liczony z wartości $f(k/n)$ dla $k = 0, \dots, n$. Żeby uniknąć przepięć przy dużych współczynnikach dwumianowych, składniki bazowe są obliczane przez logarytmy i funkcję gamma. Błąd $d_\infty(f, B_n(f))$ jest liczony tak samo jak supremum na przedziale: przez siatkę punktów na $[0, 1]$ i lokalne doszukiwanie największej wartości błędu.

```
bernstein(f, n):
    samples[k] = f(k/n) for k = 0..n

    B(x):
        result = 0
        for k = 0..n:
            log_basis =
                log(n!) - log(k!) - log((n-k)!)
                + k*log(x) + (n-k)*log(1-x)
            basis = exp(log_basis)
            correct basis at x=0 and x=1
            result += samples[k] * basis
        return result
```

Algorytm 9: Numeryczna ewaluacja wielomianu Bernsteina

```
animation_degrees(target_n, max_frames):
    if target_n <= max_frames:
        return [1,2,...,target_n]
    else:
        return max_frames degrees evenly spaced from 1 to target_n
```

Algorytm 10: Animacja Bernsteina dla dużego n

Dlaczego nie rozwijamy symbolicznie dużego B_n ?

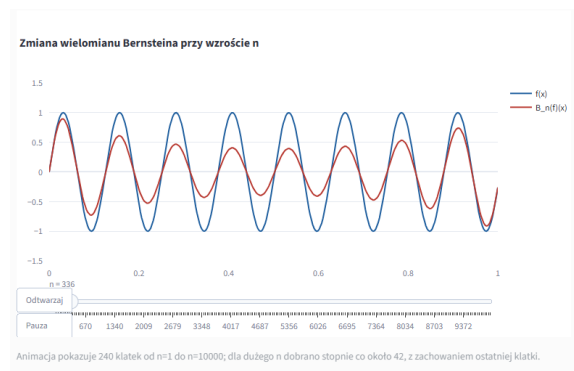
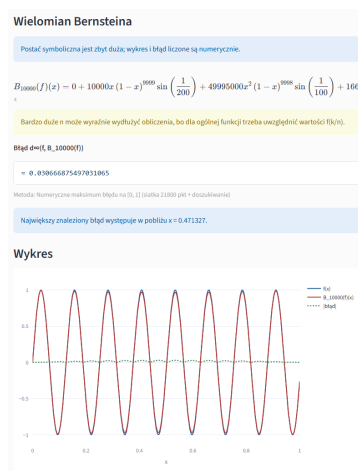
Dla $n = 10000$ pełna suma ma 10001 składników, więc jej wypisanie nie pomaga w zrozumieniu wyniku. Program pokazuje początek wzoru, ale wykres, błąd i animację liczy numerycznie.

7.3 Przykład

Jako przykład funkcji o większych oscylacjach można wziąć

$$f(x) = \sin(50x)$$

i porównać ją z wielomianem Bernsteina dla dużego stopnia, np. $n = 10000$. W takim przypadku nie ma sensu wypisywać pełnego wzoru symbolicznego, ale wykres i błąd można obliczać numerycznie. Animacja pokazuje wtedy wybrane stopnie od 1 do 10000, dzięki czemu widać, jak przybliżenie poprawia się wraz ze wzrostem n .



Animacja kolejnych wielomianów.

Wykres funkcji i wielomianu Bernsteina.

Rysunek 6: Dwa przykładowe wyniki działania modułu.

8 Podprogram 5: przeciwobraz funkcji skalarnej

8.1 Opis zadania

Dana jest funkcja

$$f : \mathbb{R}^2 \rightarrow \mathbb{R},$$

zbiór $A \subseteq \mathbb{R}$ oraz punkt $(x_0, y_0) \in \mathbb{R}^2$. Program sprawdza, czy

$$(x_0, y_0) \in f^{-1}(A),$$

czyli czy

$$f(x_0, y_0) \in A.$$

Oprócz tego aplikacja rysuje przybliżony przeciwobraz

$$f^{-1}(A) = \{(x, y) \in \mathbb{R}^2 : f(x, y) \in A\}.$$

8.2 Metoda

Zbiór A można podać jako przedział, sumę przedziałów, przecięcie przedziałów albo zbiór skończony. Najpierw program oblicza wartość $f(x_0, y_0)$ w badanym punkcie i sprawdza, czy należy ona do A . Jeżeli da się to zrobić symbolicznie, używany jest zapis dokładny; w przeciwnym razie program przechodzi do sprawdzenia numerycznego.

Rysunek jest wykonywany w wybranym oknie widoku. Program tworzy siatkę punktów, liczy na niej wartości $f(x, y)$ i koloruje te punkty, dla których $f(x, y) \in A$. Dla przedziałów rysowane są także poziomice odpowiadające końcom przedziału. Brzeg przedziału domkniętego jest linią ciągłą, a brzeg przedziału otwartego linią przerywaną. Dla zbiorów skończonych rysowane są poziomice $f(x, y) = a$ dla kolejnych elementów $a \in A$.

```
scalar_preimage(f, A, P=(x0,y0)):  
    value = f(x0,y0)  
    try symbolic membership: value in A  
    if symbolic result is undecidable:  
        classify value numerically  
  
    create grid in chosen drawing window  
    Z = f(X,Y)  
    mask = membership of Z in A  
    draw filled contour of mask  
    draw boundary contours f(x,y)=endpoint(A)  
    use dashed line for open endpoints
```

Algorytm 11: Przynależność punktu i rysowanie przeciwobrazu

Interpretacja rysunku

Kolor wypełnienia oznacza punkty, które spełniają warunek $f(x, y) \in A$ w wybranym oknie. Linie brzegowe są poziomiceami funkcji f , więc pokazują, gdzie wartość funkcji trafia w granice zbioru A .

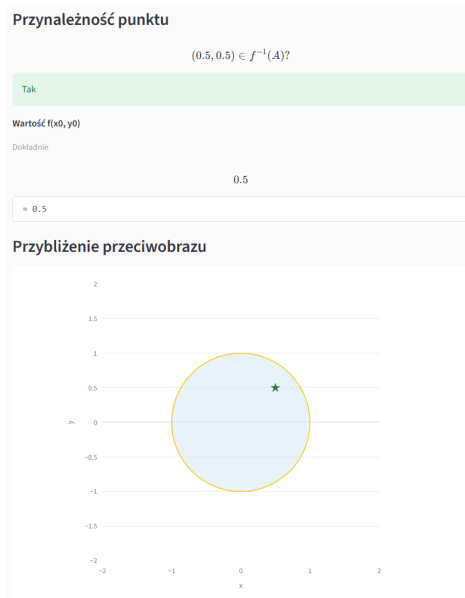
8.3 Przykład

Jeżeli

$$f(x, y) = x^2 + y^2, \quad A = [0, 1],$$

to przeciwobrazem jest domknięty dysk jednostkowy

$$f^{-1}(A) = \{(x, y) : x^2 + y^2 \leq 1\}.$$



Rysunek 7: Przeciwobraz zbioru dla funkcji $f : \mathbb{R}^2 \rightarrow \mathbb{R}$.

9 Podprogram 6: odwzorowania wektorowe

9.1 Opis zadania

Dane jest odwzorowanie

$$\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}^2$$

oraz zbiory $B, C \subseteq \mathbb{R}^2$. Program rysuje przybliżony przeciwobraz

$$\Phi^{-1}(B) = \{(x, y) : \Phi(x, y) \in B\}$$

oraz obraz

$$\Phi(C) = \{\Phi(x, y) : (x, y) \in C\}.$$

9.2 Metoda

Przeciwobraz jest liczony przez sprawdzanie warunku przynależności $\Phi(x, y) \in B$ na siatce punktów w wybranym oknie. Jeżeli B jest opisany relacją, np. $u^2 + v^2 \leq 1$, program podstawia do tej relacji współrzędne odwzorowania $\Phi_1(x, y)$ i $\Phi_2(x, y)$, a następnie rysuje obszar oraz brzeg przez poziomice odpowiedniej różnicy stron równania.

Obraz zbioru C jest rysowany przez próbkowanie punktów należących do C i nanoszenie wartości $\Phi(x, y)$ na płaszczyznę obrazu. Dla odwzorowania identycznościowego program korzysta z dokładniejszego trybu rysowania samego zbioru i jego brzegu, ponieważ wtedy obraz jest po prostu tym samym zbiorem. Dla ogólnego odwzorowania używane jest próbkowanie, dlatego jakość zależy od wybranej rozdzielczości.

Program obsługuje zbiory opisane nierównościami, prostokąty, relacje oraz proste operacje na zbiorach, takie jak suma i przecięcie. Przy przecięciach brzeg jednego warunku jest przycinany drugim warunkiem, dzięki czemu na rysunku nie pojawiają się zbędne fragmenty brzegu poza właściwym zbiorem. Dla nierówności ostrych brzeg jest rysowany linią przerywaną, a dla nieostrych linią ciągłą. Użytkownik może zwiększyć jakość rysowania, jeżeli zależy mu na dokładniejszym obrazie kosztem czasu obliczeń.

```
vector_preimage(Phi, B, window):
    create grid (X,Y) in source window
    U = Phi_1(X,Y)
    V = Phi_2(X,Y)
    mask = membership of (U,V) in B
    draw filled contour of mask

    if B is relation lhs(u,v) <= rhs(u,v):
        substitute u=Phi_1(x,y), v=Phi_2(x,y)
        draw boundary as contour lhs-rhs = 0
```

Algorytm 12: Przeciwobraz w module odwzorowań wektorowych

```
vector_image(Phi, C, window):
    create grid in source window
    mask = membership of (X,Y) in C
    U = Phi_1(X,Y)
    V = Phi_2(X,Y)
    keep only points where mask is true and U,V are finite
    if too many points:
```

```
thin the sample
draw points (U,V) in target plane
```

Algorytm 13: Obraz zbioru w module odwzorowań wektorowych

Specjalny przypadek odwzorowania identycznościowego

Jeżeli $\Phi(x, y) = (x, y)$, obraz zbioru C jest równy samemu C . Program wykrywa ten przypadek i zamiast chmury punktów rysuje wypełniony obszar oraz brzeg, co daje znacznie lepszą jakość dla zbiorów zadanych nierównościami.

9.3 Przykład

Przykładowo można użyć odwzorowania

$$\Phi(x, y) = \left(x \cos(x^2 + y^2) - y \sin(x^2 + y^2), x \sin(x^2 + y^2) + y \cos(x^2 + y^2) \right),$$

czyli obrotu punktu o kąt zależny od jego odległości od początku układu. Dla zbioru

$$C = \{(x, y) : |x| \leq 1.6, |y| \leq 1.6, \sin(4x) \sin(4y) \geq 0\}$$

oraz

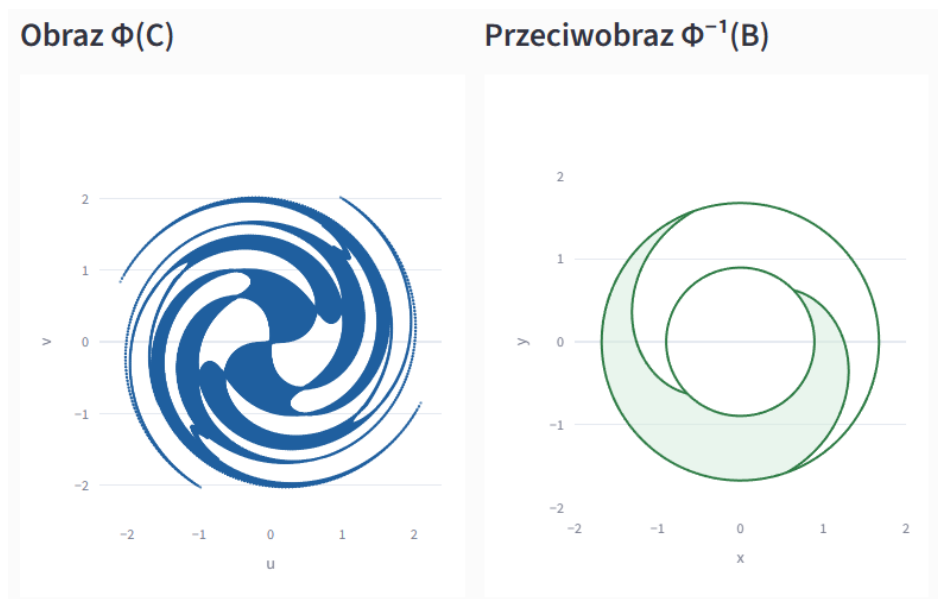
$$B = \{(u, v) : 0.8 \leq u^2 + v^2 \leq 2.8, u \geq 0\}$$

program rysuje obraz $\Phi(C)$ oraz przeciwobraz $\Phi^{-1}(B)$.

Do prostego sprawdzenia jakości rysowania można też użyć odwzorowania identycznościowego $\Phi(x, y) = (x, y)$ oraz zbioru

$$(x^2 + y^2 - 1)^3 - x^2 y^3 < 0,$$

który daje charakterystyczny kształt serca.



Rysunek 8: Obraz i przeciwobraz dla odwzorowania $\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}^2$.

10 Ograniczenia programu

Program nie próbuje rozwiązywać symbolicznie wszystkich możliwych problemów z analizy i topologii. Założono, że użytkownik podaje funkcje i zbiory w rozsądnej postaci: bez niejednoznacznych zapisów, z odpowiednio dobranym oknem rysowania oraz z dziedziną zgodną z treścią zadania.

Najważniejsze ograniczenia są następujące:

- dokładne wyniki są pokazywane tam, gdzie SymPy jest w stanie je sensownie uprościć,
- supremum dla trudnych funkcji jest liczone numerycznie,
- rysunki przeciwobrazów i obrazów są przybliżeniami zależnymi od siatki oraz wybranego okna,
- bardzo skomplikowane wzory mogą wymagać mniejszej rozdzielczości rysowania albo dłuższego czasu obliczeń,
- program zakłada, że funkcje podane w zadaniach o supremum są ciągłe na wybranych dziedzinach.